

Объектно- Ориентированное Программирование

(МИСиС, зима-весна 2026)

Полевой Дмитрий Валерьевич

к.т.н., доцент КиК

teams: 8url2s2

e-mail: dvpsun@gmail.com

tg: @dvpsun

Зачем «много знать и уметь»

- использовать чужое
- изобретать новое (когда надо), а не повторять
- оценивать `+` и `-` (знать что важно, а что нет)
- читать/искать/писать документацию
- не совершать грубых/дорогих/фатальных ошибок

Ошибки

- ошибки неизбежны – часть процесса
- хорошую версию ПО получается сделать за 3-4+ итерации выпуска
- изменения постоянны
- более успешен тот, кто «может пережить» больше ошибок и лучше адаптироваться

Программная инженерия

- ответ на вызовы все возрастающей сложности
 - развитие языков
 - развитие инструментов
 - развитие методов разработки
- как создавать надежные системы из ненадежных компонентов

Когда проявляются проблемы

- система с трудом поддается изменениям
- высокая связность
- любое минимальное изменение вызывает эффект «снежного кома», затрагивающего другие места
- изменения проявляются в местах, которые не имеют прямого отношения к изменяемому коду

О чем стоит помнить

- не существует «серебряной пули»
- существует много схожих ситуаций
- редко ситуации описаны полностью
- нехватку информации компенсируйте сами

Библиотека vs. Фреймворк

- кто кого вызывает
- кто управляет потоком исполнения
- аналогия
 - вы пишете конвейер и используете инструменты
 - конвейер готовый, вы меняете его элементы

Библиотека (library)

- основной поток исполнения ваш
- вы вызываете код библиотеки
- заменить библиотеку относительно просто
 - легкая обертка
 - переписывание кода-сервера

Фреймворк (framework)

- основной поток исполнения «не ваш»
- фреймворк вызывает ваш код
 - фреймворк определяет структуру
 - вы заполняете точки расширения
- заменить фреймворк сложно

Фреймворк (пример)

```
#include "app.hpp"
```

```
int main() {  
    App app;  
    return app.exec();    // управление фреймворку  
}
```

Фреймворк (ппример)

```
class App : public QApplication {  
public:  
    void onClick() override {  
        // фреймворк вызовет  
        // ваш код  
    }  
};
```

ваши вопросы

Объединение (union)

- хранение одного из заданных типов
- крайне опасен
 - нельзя проверить, «какой тип сейчас»
 - деструктор не вызывается автоматически
 - нет защиты от чтения/записи не того типа

Объединение (пример)

```
union Data {  
    int    i;  
    double d;  
    char   s[16];  
};
```

```
Data x;
```

```
x.i = 42;
```

```
double y = x.d;
```

`std::variant` (C++17)

- типобезопасное объединение
- хранит значение одного из заданных типов
- «знает» текущий тип
- управляет временем жизни объекта
 - конструирует
 - вызывает деструктор при смене типа значения

std::variant (пример)

```
std::variant<int, double, std::string> v;
```

```
v = 42;           // текущий int  
v = 3.14;         // текущий double  
v = "hello";      // текущий string
```


`std::variant` (получение значения)

- `std::get<N>` - по индексу типа
- `std::get_if<T>` - по типу из списка
 - через указатель с проверкой
- `std::get<T>` - по типу из списка
 - бросает исключение при неправильном типе

std::variant (пример)

```
std::variant<int, double, string> v = 42;
```

```
int n_0 = std::get<0>(v); // OK
```

```
int n_t = std::get<int>(v); // OK
```

std::variant

(пример)

```
std::variant<int, double, string> v = 42;
```

```
if (auto* p = std::get_if<int>(&v)) {  
    // *p — значение, если тип совпал  
}
```

```
double d = std::get<double>(v);  
// бросает std::bad_variant_access
```

`std::variant` (конструирование)

- **умолчательно**
 - первый тип из списка
- `std::monostate`
 - используйте первым в списке, если другие без `T ( )`
 - умолчательный тип «без значения»

`std::variant` (полезен)

- множество типов фиксировано
- типы известны на этапе компиляции
- новые типы не предполагаются
- хранение на стеке (не в куче)

`std::variant` (не подходит)

- типов много и они будут добавляться
 - лучше виртуальные функции
- нужен открытый полиморфизм
 - типы добавляет сторонний код

ваши вопросы

SOLID

(принципы)

- SRP: один класс — одна ответственность
- OCP: открыт для расширения, закрыт для изменения
- LSP: подтип годится везде, где ожидается базовый тип
- ISP: много узких интерфейсов, чем один широкий
- DIP: зависеть от абстракций, не от конкретных классов

Single Responsibility Principle (SRP)

- класс должен иметь только ответственность
- ответственность должна быть полностью инкапсулирована в класс
- поведения должны быть направлены исключительно на обеспечение этой ответственности

Single Responsibility Principle

(плохо)

класс знает и о данных, и о сериализации

```
class User {  
    void saveToDatabase() { ... }  
    void sendEmail() { ... }  
};
```

Single Responsibility Principle

(хорошо)

каждый класс — одна задача

```
class User          { /* только данные */ };  
class UserRepo      { void save(User&); };  
class EmailSender{ void send(User&); };
```

Single Responsibility Principle

(проблемы)

- «одна ответственность» — что считать одной ответственностью?
- можно делить классы по-разному и ссылаться на SRP
- ведёт к фрагментации: десятки мелких классов
- логика размазана по файлам — сложнее понять поток выполнения

Open/Closed Principle (OCP)

- открыт для расширения, закрыт для изменения
- новое поведение - через новый класс, не правку старого

Open/Closed Principle

(плохо)

добавление фигуры → правка существующего кода

```
double area(Shape* s) {  
    if (s->type == CIRCLE)      return ...;  
    if (s->type == RECTANGLE) return ...;  
}
```

Open/Closed Principle

(хорошо)

новая фигура → новый класс

```
struct IShape {  
    virtual double area() = 0;  
};
```

Open/Closed Principle

(хорошо)

```
struct Circle : IShape {  
    double area() override;  
};
```

```
struct Rectangle : IShape {  
    double area() override;  
};
```


Open/Closed Principle

(критика)

- провоцирует преждевременную абстракцию - расширяемость закладывается до того, как ясно, что именно будет меняться
- в C++ OCP реализуется не только наследованием, но и шаблонами
- строгая трактовка через виртуальные классы устарела

Liskov Substitution Principle (LSP)

- подтип должен быть полностью заменяем базовым типом.
- если В наследует А — везде, где ожидается А, можно передать В

Liskov Substitution Principle

(критика)

- наследование часто нарушает реальные инварианты предметной области
- предпочитать композицию наследованию

ваши вопросы

Interface Segregation Principle (ISP)

- лучше много узких интерфейсов, чем один широкий
- клиент не должен зависеть от методов, которые не использует

Interface Segregation Principle

(плохо)

```
struct IDevice {  
    virtual void print() = 0;  
    virtual void scan() = 0;  
    virtual void fax() = 0;  
};
```

Interface Segregation Principle (хорошо)

```
struct IPrinter { virtual void print() = 0; };  
struct IScanner { virtual void scan() = 0; };  
  
struct MultiFunctionDevice  
    : IPrinter, IScanner { ... };  
struct SimplePrinter  
    : IPrinter { ... };
```

Interface Segregation Principle

(проблемы)

- Слишком дробные интерфейсы усложняют понимание архитектуры

Dependency Inversion Principle (DIP)

- зависеть от абстракций, не от конкретных классов
- детали реализации зависят от интерфейсов — не наоборот

Dependency Inversion Principle

(плохо - Navigator жёстко привязан к CarRouter)

```
class Navigator {  
    CarRouter r_;  
public:  
    void build_r(Pt a, Pt b) {  
        r_.route(a, b); }  
};
```

Dependency Inversion Principle

(хорошо - зависимость от абстракции)

```
class Navigator {  
    unique_ptr<IRouter> r_;  
public:  
    explicit Navigator(unique_ptr<IRouter> r)  
        : r_(move(r)) {}  
    void build_r(Pt a, Pt b) { r_>route(a, b); }  
};
```

Dependency Inversion Principle

(критика)

- накладные расходы: вместо прямого вызова
- `virtual + unique_ptr`

Критика SOLID

- SOLID формулированы в контексте Java
- в C++
 - есть шаблоны, концепты, `std::function`
 - многие проблемы решаются без иерархий классов

ваши вопросы

Strategy

(паттерн)

- алгоритм должен вести себя по-разному в зависимости от условий
- поведение нужно менять в runtime

примеры:

- расчет маршрута: пешком/авто/велосипед и т.д.
- отрисовка фигуры: круг/прямоугольник/ромб и т.д.

Наивное решение

```
class Navigator {  
public:  
    void buildRoute(Pt from, Pt to, int tr) {  
        // ...  
    };  
    // ...  
};
```


Наивное решение

```
void buildRoute(Pt from, Pt to, int tr) {  
    if (tr == 0) {  
        // пешеход: дорожки, светофоры и т.п  
        ...  
    } else if (tr == 1) {  
        // авто: объезд пробок, платные дороги  
        ...  
    }  
    // новый транспорт → снова правим  
}
```

Strategy

(паттерн - проблемы)

- добавление варианта → правка функции
- растущая цепочка `if/else` или `switch`
- сложно
 - читать
 - тестировать
 - командно разрабатывать

Strategy

(паттерн - идея)

- вынести алгоритм в объект
- контекст использует интерфейс
- контекст и алгоритм независимы
- новый алгоритм — новый класс

Варианты реализации

- виртуальный вызов
- функтор (callable object)

Интерфейс

```
struct IRouteS {  
    virtual void buildRoute(Pt from, Pt to) = 0;  
    virtual ~IRouteS() = default;  
};
```

Реализация интерфейса

```
struct CarStrategy : IRouteS {  
    void build_r(Pt from, Pt to) override {  
        // объезд пробок, платные дороги  
    }  
};
```

Контекст

```
class Navigator {  
    std::unique_ptr<IRoutes> s_;  
public:  
    explicit Navigator(unique_ptr<IRoutes> s)  
        : s_(std::move(s)) {}  
};
```

Контекст

```
class Navigator {  
public:  
    void build_r(Pt from, Pt to) {  
        s_->build_r(from, to);  
    }  
};
```


Использование

```
Navigator nav(CarStrategy{});  
nav.build_r(a, b);
```

Интерфейс (функтор)

```
std::function<void(Pt, Pt)>
```

Реализация интерфейса (функтор)

```
auto walk_s = [] (Pt a, Pt b) {  
    // пешеходные дорожки, светофоры  
};
```

```
auto car_s = [] (Pt a, Pt b) {  
    // объезд пробок  
};
```

КОНТЕКСТ

```
class Navigator {  
    function<void(Pt, Pt)> s_;  
public:  
    explicit Navigator(function<void(Pt, Pt)> s)  
        : s_(move(s)) {}  
};
```

КОНТЕКСТ

```
void Navigator::set_s(  
    function<void(Pt, Pt)> s) {  
    s_ = move(s);  
}  
  
void Navigator::build_r(Pt from, Pt to) {  
    s_(from, to);  
}
```

Использование

```
Navigator nav(walk_s);
```

```
nav.build_r(a, b);
```

```
nav.set_s(car_s);
```

```
nav.build_r(a, b);
```

Тип для объединения стратегий

```
using RouteStrategy = variant<
    WalkStrategy,
    CarStrategy,
    BikeStrategy
>;
```

КОНТЕКСТ

```
void build_r(Pt from, Pt to) {  
    visit([&](auto& s) {  
        s.build_r(from, to);  
    }, s_);  
}
```


Типичные ошибки

- стратегия хранит состояние контекста - нарушение инкапсуляции
- слишком мелкие стратегии - накладные расходы превышают пользу
- использование `new` / сырых указателей вместо `unique_ptr`
- забытый виртуальный деструктор в интерфейсе

Рекомендации

(когда использовать)

- выбирать/менять алгоритм в runtime
- есть 3+ вариантов одного поведения
- параллельная разработка реализаций стратегии
- независимое тестирование алгоритмов без контекста

Рекомендации

(когда избегать)

- алгоритм один и никогда не меняется
- вариантов два — достаточно простого флага или шаблонного параметра
- производительность критична → function имеет накладные расходы

ваши вопросы